

Nombre: Luis de la Garza González

Matrícula: al03101869

Nombre del curso: Machine Learning.

Trabajo: Actividad 03.- Cálculo
Vectorial y Probabilidades.

Nombre del profesor: Zuriel Dathan Mora Felix.

Fecha: 28 de septiembre de 2025

Contenido

Objetivo	3
Instrucciones:	3
Desarrollo	5
1. Derivadas	5
2. Derivadas parciales en Python	7
3. Investigación sobre diferentes implementaciones del algoritmo de propagación inversa y construcción de un ejemplo en Python donde se compruebe su funcionamiento.	9
4. Cálculo de Probabilidad de Falla	12
5. Cálculos con Forest Fires Dataset.	13
Conclusiones	17
Liga al código en Github	18

Objetivo

Aplicar técnicas de diferenciación y análisis de funciones utilizando Python en Jupyter Notebook, integrando conceptos de cálculo y probabilidad.

Instrucciones:

Utiliza un cuaderno de Jupyter Notebook y el lenguaje de programación Python para realizar las siguientes operaciones:

1. Derivadas

Primera derivada de las siguientes funciones:

- A. $f(x) = 3x + 4e^{-4x}$
- B. $f(x) = 5x^{(3x+1)} - 8x^2 - 3x$

Segunda derivada de las siguientes funciones:

- A. $f(x) = 4x^4 + 6x^2 + e^{(4x)^2}$
- B. $f(x) = 1/(2x^3) + 8x$

2. .

Calcula las primeras derivadas parciales de las siguientes funciones:

- A. $f(x,y) = 4x^3y + 5y^2 - 7x$
- B. $f(x,y) = y^2 / (1 - e^x)$

Calcula las segundas derivadas de las siguientes funciones:

- A. $f(x,y) = 3x^5y + y^3 / 3$
- B. $f(x,y) = \sin^{-1} x^3 / (\cos 2y)^2$

3. Investiga sobre diferentes implementaciones del algoritmo de propagación inversa y construye un ejemplo en Python donde se compruebe su funcionamiento.

4. Dada la siguiente situación:

Los dispositivos electrónicos contruidos con transistores en la década de 1990 presentaban dos fallas principales: la fuente de alimentación y el circuito principal (tarjeta principal). Según las estadísticas de esa época, un taller promedio recibía de manera mensual un 30% de equipos por fallas en la fuente de alimentación y la efectividad de reparación era del 75% si la falla estaba en la tarjeta principal.

- Calcula la probabilidad de que un equipo que haya ingresado al taller por problemas en la tarjeta principal.
- Implementa tu solución en un cuaderno de Jupyter Notebook utilizando Python.

5. Utilizando un cuaderno de Jupyter Notebook y el lenguaje de programación Python, realiza las siguientes operaciones:

- Importa a tu espacio de trabajo el siguiente dataset: Forest Fires Dataset en Kaggle
- Crea un arreglo que contenga los valores de la columna etiquetada por: #temp
- Construye el histograma referente al arreglo creado en el punto anterior.
- Evalúa si la gráfica obtenida tiene semejanzas con la distribución normal y obtén los siguientes estadísticos: Media, Mediana y Desviación estándar.
- Dibuja la gráfica de la distribución gaussiana inferida por los datos, en caso de que corresponda, y evalúa la suposición realizada en el punto anterior.

Desarrollo

Utilizando un cuaderno de Jupyter Notebook y el lenguaje de programación Python realiza las siguientes operaciones:

1. Derivadas

a) Primera derivada de las siguientes funciones:

A. $f(x) = 3x + 4e^{-(4x)}$

B. $f(x) = 5x^{(3x+1)} - 8x^2 - 3x$



```
In [1]: # C:\Users\luisg\OneDrive\Tec Milenio\Master en IA\002 Machine Learning\Tareas\Actividad-003\Actividad-03.ipynb
import sympy as sp

# Definir la variable simbólica
x = sp.Symbol('x')

# Función 1: f(x) = 3x + 4e^{-4x}
f1 = 3*x + 4*sp.exp(-4*x)
df1 = sp.diff(f1, x)

# Función 2: f(x) = 5x^{(3x + 1)} - 8x^2 - 3x
f2 = 5*x**(3*x + 1) - 8*x**2 - 3*x
df2 = sp.diff(f2, x)

# Mostrar resultados
print("Primera derivada de f1(x):", df1)
print("Primera derivada de f2(x):", df2)

Primera derivada de f1(x): 3 - 16*exp(-4*x)
Primera derivada de f2(x): -16*x + 5*x**(3*x + 1)*(3*log(x) + (3*x + 1)/x) - 3
```

b) Segunda derivada de las siguientes funciones:

A. $f(x) = 4x^4 + 6x^2 + e^{(4x)^2}$

B. $f(x) = 1/(2x^3) + 8x$

```
In [9]: # Calcular las segundas derivadas de ambas funciones:

import sympy as sp

# Definir la variable simbólica
x = sp.Symbol('x')

# Función A:  $f(x) = 4x^4 + 6x^2 + e^{(4x)^2}$ 
fA = 4*x**4 + 6*x**2 + sp.exp(4*x**2)
segunda_derivada_A = sp.diff(fA, x, 2)

# Función B:  $f(x) = 1/(2x^3) + 8x$ 
fB = 1 / (2*x**3) + 8*x
segunda_derivada_B = sp.diff(fB, x, 2)

# Mostrar resultados

print("Segunda derivada de fA(x): ", segunda_derivada_A)
# sp.pprint(segunda_derivada_A)

print("\nSegunda derivada de fB(x): ", segunda_derivada_B)
# sp.pprint(segunda_derivada_B)

Segunda derivada de fA(x): 4*(16*x**2*exp(4*x**2) + 12*x**2 + 2*exp(4*x**2) + 3)

Segunda derivada de fB(x): 6/x**5
```

In []:

2. Derivadas parciales en Python

Calcula las primeras derivadas parciales de las siguientes funciones:

- A. $f(x, y) = 4x^3y + 5y^2 - 7x$
- B. $f(x, y) = \frac{y^2}{1-e^x}$

```
In [20]: # Derivadas Parciales en Python
import sympy as sp

# Definir variables simbólicas
x, y = sp.symbols('x y')

# Función A
fA = 4*x**3*y + 5*y**2 - 7*x
dFA_dx = sp.diff(fA, x)
dFA_dy = sp.diff(fA, y)

# Función B
fB = y**2 / (1 - sp.exp(x))
dFB_dx = sp.diff(fB, x)
dFB_dy = sp.diff(fB, y)

# Mostrar resultados
print("Derivadas parciales de fA(x, y):")
sp.pprint(dFA_dx)
sp.pprint(dFA_dy)

print("\nDerivadas parciales de fB(x, y):")
sp.pprint(dFB_dx)
sp.pprint(dFB_dy)
```

Derivadas parciales de fA(x, y):

Para mayor claridad, presentamos las ecuaciones resultantes de las primeras derivadas parciales:

Función A

- $f(x, y) = 4x^3y + 5y^2 - 7x$

Derivadas parciales:

Respecto a x : $\partial f / \partial x = 12x^2y - 7$

Respecto a y : $\partial f / \partial y = 4x^3 + 10y$

Función B

- $f(x, y) = y^2 / 1 - e^x$

Derivadas parciales:

Respecto a x : $\partial f / \partial x = y^2 e^x / (1 - e^x)^2$

Respecto a y : $\partial f / \partial y = 2y / (1 - e^x)$

Calcula las segundas derivadas parciales de las siguientes funciones:

A.

$$f(x, y) = 3x^5y + \frac{y^3}{3}$$

B.

$$f(x, y) = \sin^{-1}\left(\frac{x^3}{(\cos 2y)^2}\right)$$

```
In [21]: # Segundas derivadas Parciales en Python
import sympy as sp

x, y = sp.symbols('x y')
fA = 3*x**5*y + y**3/3

# Primeras derivadas
fx_A = sp.diff(fA, x)
fy_A = sp.diff(fA, y)

# Segundas derivadas
fxx_A = sp.diff(fx_A, x)
fyy_A = sp.diff(fy_A, y)
fxy_A = sp.diff(fx_A, y)
fyx_A = sp.diff(fy_A, x)

# Mostrar resultados
sp.pprint(fxx_A) # ∂²f/∂x²
sp.pprint(fyy_A) # ∂²f/∂y²
sp.pprint(fxy_A) # ∂²f/∂x∂y
sp.pprint(fyx_A) # ∂²f/∂y∂x
```

```
3
60·x³·y
2·y
4
15·x³
4
15·x³
```

Los resultados son:

- $\frac{\partial^2 f}{\partial x^2} = 60x^3y$
- $\frac{\partial^2 f}{\partial y^2} = 2y$
- $\frac{\partial^2 f}{\partial x \partial y} = 15x^4$
- $\frac{\partial^2 f}{\partial y \partial x} = 15x^4$

3. Investigación sobre diferentes implementaciones del algoritmo de propagación inversa y construcción de un ejemplo en Python donde se compruebe su funcionamiento.

La propagación inversa es un proceso clave en el entrenamiento de redes neuronales. Se basa en el principio de optimización por descenso de gradiente y permite ajustar los pesos y sesgos de las neuronas en cada capa de la red. Este proceso se divide en dos fases: la propagación hacia adelante, donde se calculan las salidas y se compara con las salidas deseadas, y la propagación hacia atrás, donde se calculan los gradientes del error y se utilizan para actualizar los pesos y sesgos. La retropropagación es fundamental para aprender representaciones complejas y mejorar la precisión de las predicciones en redes neuronales profundas.

Los tipos de propagación inversa en inteligencia artificial incluyen:

- **Propagación inversa:** Es el algoritmo de entrenamiento más común para redes neuronales, que permite ajustar los pesos del modelo para minimizar la pérdida.
- **Redes neuronales recurrentes (RNN):** Utilizan datos de series temporales y tienen memoria de lo que sucedió en capas anteriores, lo que les permite procesar información en tiempo real.
- **Redes neuronales de memoria a largo/corto plazo (LSTM):** Son una forma avanzada de RNN que puede recordar información a lo largo de varias capas, útil en aplicaciones como el reconocimiento de voz.
- **Propagación inversa en Python:** Se implementa para entrenar redes neuronales, donde se calculan los gradientes y se ajustan los pesos para reducir el error.
- **Con momento:** Añade un término que suaviza el aprendizaje y evita oscilaciones
- **Con regularización:** Penaliza pesos grandes para evitar sobreajuste
- **Método de corrección de errores:** Comienza con el resultado de la red neuronal y realiza el proceso en sentido inverso para llegar al principio, detectando errores para mejorar la exactitud.

Ejemplo en Python:

```
In [14]: # Ejemplo en Python de algoritmo de propagación inversa donde se compruebe su funcionamiento.
import numpy as np

# Define La función de activación sigmoide, que transforma cualquier número en un valor entre 0 y 1.
# Se usa para introducir no linealidad en la red.
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Define La derivada de La función sigmoide, necesaria para calcular Los gradientes durante La propagación inversa.
def sigmoid_deriv(x):
    return x * (1 - x)

# Datos de entrada (2 neuronas, 4 muestras)
# Son Las combinaciones posibles de dos bits
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

# Salidas esperadas (XOR)
# Este es el resultado del XOR lógico: solo es 1 cuando Los bits son diferentes.
y = np.array([[0], [1], [1], [0]])

# Inicialización de pesos
np.random.seed(1)
# W1 conecta Las 2 entradas con 4 neuronas ocultas.
W1 = np.random.rand(2, 4) # pesos capa entrada → oculta
# W2 conecta Las 4 neuronas ocultas con 1 salida.
W2 = np.random.rand(4, 1) # pesos capa oculta → salida

# Entrenamiento, se repite 50,000 veces para que La red aprenda.
for epoch in range(50000):
    # Propagación hacia adelante
    # Multiplica entradas por pesos (X · W1) y aplica La función sigmoide
    hidden_input = np.dot(X, W1)
    hidden_output = sigmoid(hidden_input)

    # Calcula La salida final de La red:
    # Multiplica salidas ocultas por pesos (hidden_output · W2)
    # Aplica sigmoide para obtener La predicción
    final_input = np.dot(hidden_output, W2)
    final_output = sigmoid(final_input)

    # Calcula el error entre La salida esperada "y" y La salida obtenida
    error = y - final_output

    # Propagación inversa
    # Calcula el gradiente de La salida usando La derivada de La sigmoide
    d_output = error * sigmoid_deriv(final_output)
    # Propaga el error hacia atrás a La capa oculta usando La regla de La cadena
    d_hidden = d_output.dot(W2.T) * sigmoid_deriv(hidden_output)

    # Actualiza Los pesos usando Los gradientes calculados. Esto es el corazón del aprendizaje.
    W2 += hidden_output.T.dot(d_output)
    W1 += X.T.dot(d_hidden)

    # Imprimimos La salida en La enésima época para ver su evolución.
    # Debería aproximarse a [0, 1, 1, 0].
    print("Salida final en la época:", epoch)
    print(final_output)

# Resultados finales
print("Resultados esperados:")
print(y)
```

Ejecutamos la celda con 50,000 épocas y obtenemos los siguientes resultados:

```
Salida final en la época: 0
[[0.73497243]
 [0.75492189]
 [0.76945373]
 [0.78668195]]
Salida final en la época: 1
[[0.69155319]
 [0.70849376]
 [0.72111898]
 [0.73632909]]
Salida final en la época: 2
[[0.64827323]
 [0.66195116]
 [0.67197968]
 [0.68461396]]
.
.
.
Salida final en la época: 49998
[[0.00614437]
 [0.99346787]
 [0.99366943]
 [0.00636152]]
Salida final en la época: 49999
[[0.0061443 ]
 [0.99346794]
 [0.99366949]
 [0.00636146]]
Resultados esperados:
[[0]
 [1]
 [1]
 [0]]
Salida final después del entrenamiento:
[[0.0061443 ]
 [0.99346794]
 [0.99366949]
 [0.00636146]]
```

Podemos ver como en cada época del ciclo, la salida final se va aproximando a los resultados esperados.

4. Cálculo de Probabilidad de Falla

Dada la siguiente situación:

Los dispositivos electrónicos que se construían a base de transistores en la década de 1990 presentaban dos tipos de fallas principales: la fuente de alimentación y el circuito principal (tarjeta principal). Según las estadísticas de esa época, un taller promedio recibía de manera mensual un 30% de equipos con afectaciones en la fuente de alimentación, y la efectividad de la reparación de un equipo era de un 75% si la falla estaba localizada en el circuito principal.

Calcula la probabilidad de que un equipo que no tuviera reparación hubiera ingresado al taller por problemas en la tarjeta principal. Implementa tu respuesta mediante código en un Jupyter Notebook de Python.

Datos del problema

$$P(\text{Fuente}) = 0.30$$

$$P(\text{Tarjeta}) = 0.70 \text{ (complemento)}$$

$$P(\text{Reparacion} \mid \text{Tarjeta}) = 0.75$$

Entonces

$$P(\text{No reparación} \mid \text{Tarjeta}) = 0.25$$

Queremos calcular:

$$P(\text{Tarjeta} \mid \text{No reparación}) = P(\text{Tarjeta} \cap \text{No reparación}) / P(\text{No reparación})$$

Implementación de la solución en un cuaderno de Jupyter Notebook utilizando Python.

```
In [19]: # Probabilidad de que el equipo haya ingresado al taller por falla en tarjeta dado que no ha sido reparado
# Probabilidades dadas
P_fuente = 0.30
P_tarjeta = 0.70
P_no_reparacion_dado_tarjeta = 0.25

# Probabilidad conjunta: no reparación y tarjeta
P_no_reparacion_y_tarjeta = P_tarjeta * P_no_reparacion_dado_tarjeta

# Suposición: Los equipos con falla en fuente no se reparan (no se especifica lo contrario)
P_no_reparacion_dado_fuente = 1.0
P_no_reparacion_y_fuente = P_fuente * P_no_reparacion_dado_fuente

# Probabilidad total de no reparación
P_no_reparacion = P_no_reparacion_y_tarjeta + P_no_reparacion_y_fuente

# Probabilidad condicional
P_tarjeta_dado_no_reparacion = P_no_reparacion_y_tarjeta / P_no_reparacion

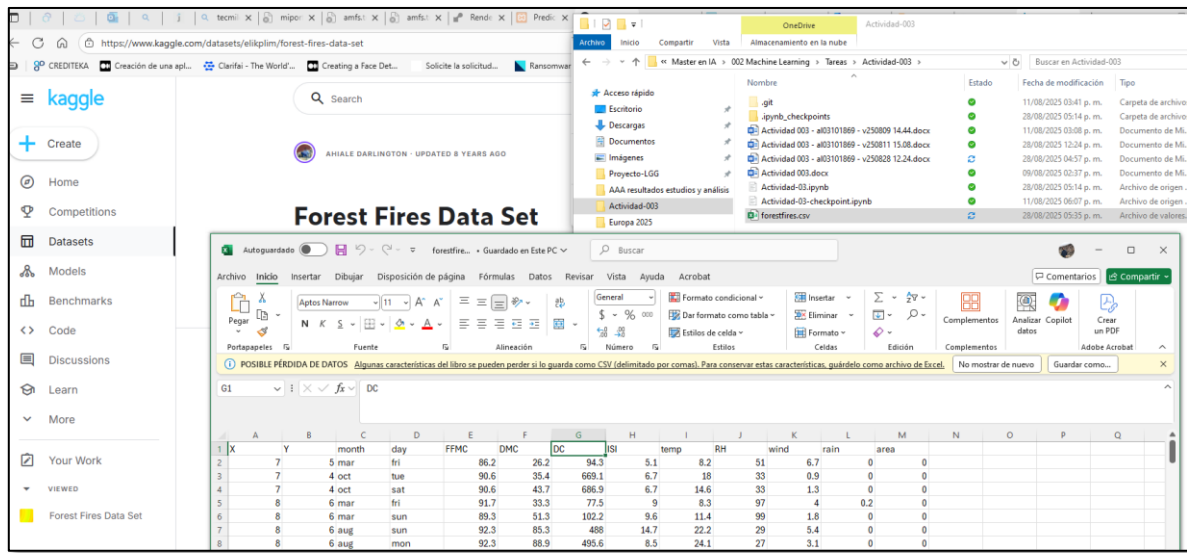
print(f"La probabilidad de que un equipo no reparado haya tenido falla en la tarjeta principal es: {P_tarjeta_dado_no_reparacion}")

La probabilidad de que un equipo no reparado haya tenido falla en la tarjeta principal es: 0.3684
```

5. Cálculos con Forest Fires Dataset.

Utilizando un cuaderno de Jupyter Notebook y el lenguaje de programación Python, realiza las siguientes operaciones:

- Importamos al espacio de trabajo el dataset: Forest Fires Dataset en Kaggle



The screenshot shows a Jupyter Notebook interface with the Kaggle Forest Fires Data Set page open in the background. The foreground displays a preview of the dataset in a spreadsheet format. The data includes columns for X, Y, month, day, FFC, DMC, DC, ISI, temp, RH, wind, rain, and area.

	X	Y	month	day	FFC	DMC	DC	ISI	temp	RH	wind	rain	area
0	7	5	mar	fri	86.2	26.2	94.3	5.1	8.2	51	6.7	0.0	0.0
1	7	4	oct	tue	90.6	35.4	669.1	6.7	18.0	33	0.9	0.0	0.0
2	7	4	oct	sat	90.6	43.7	686.9	6.7	14.6	33	1.3	0.0	0.0
3	8	6	mar	fri	91.7	33.3	77.5	9.0	8.3	97	4.0	0.2	0.0
4	8	6	mar	sun	89.3	51.3	102.2	9.6	11.4	99	1.8	0.0	0.0

```
In [8]: import pandas as pd

# Cargar el dataset
df = pd.read_csv('forestfires.csv')

# Mostrar las primeras filas para verificar
df.head()
```

Out[8]:

	X	Y	month	day	FFC	DMC	DC	ISI	temp	RH	wind	rain	area
0	7	5	mar	fri	86.2	26.2	94.3	5.1	8.2	51	6.7	0.0	0.0
1	7	4	oct	tue	90.6	35.4	669.1	6.7	18.0	33	0.9	0.0	0.0
2	7	4	oct	sat	90.6	43.7	686.9	6.7	14.6	33	1.3	0.0	0.0
3	8	6	mar	fri	91.7	33.3	77.5	9.0	8.3	97	4.0	0.2	0.0
4	8	6	mar	sun	89.3	51.3	102.2	9.6	11.4	99	1.8	0.0	0.0

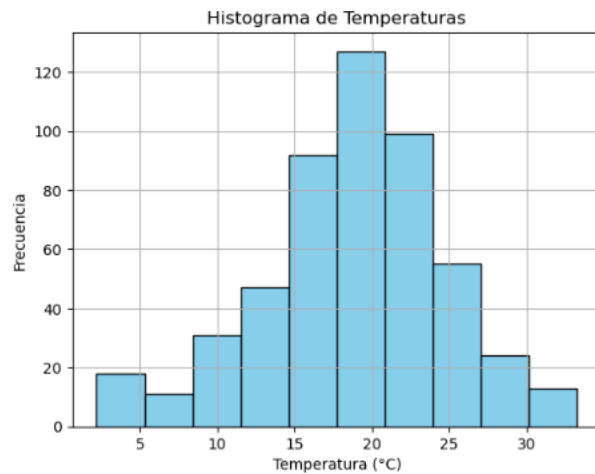
In []:

- Creamos un arreglo que contenga los valores de la columna etiquetada por: #temp

```
In [9]: import numpy as np  
  
# Extraer la columna de temperatura  
temp_array = df['temp'].values
```

- Construimos el histograma referente al arreglo creado en el punto anterior.

```
In [10]: # Construir el histograma  
  
import matplotlib.pyplot as plt  
  
plt.hist(temp_array, bins=10, color='skyblue', edgecolor='black')  
plt.title('Histograma de Temperaturas')  
plt.xlabel('Temperatura (°C)')  
plt.ylabel('Frecuencia')  
plt.grid(True)  
plt.show()
```



```
In [ ]:
```

- Evaluamos si la gráfica obtenida tiene semejanzas con la distribución normal y obtenemos los siguientes estadísticos: Media, Mediana y Desviación estándar.

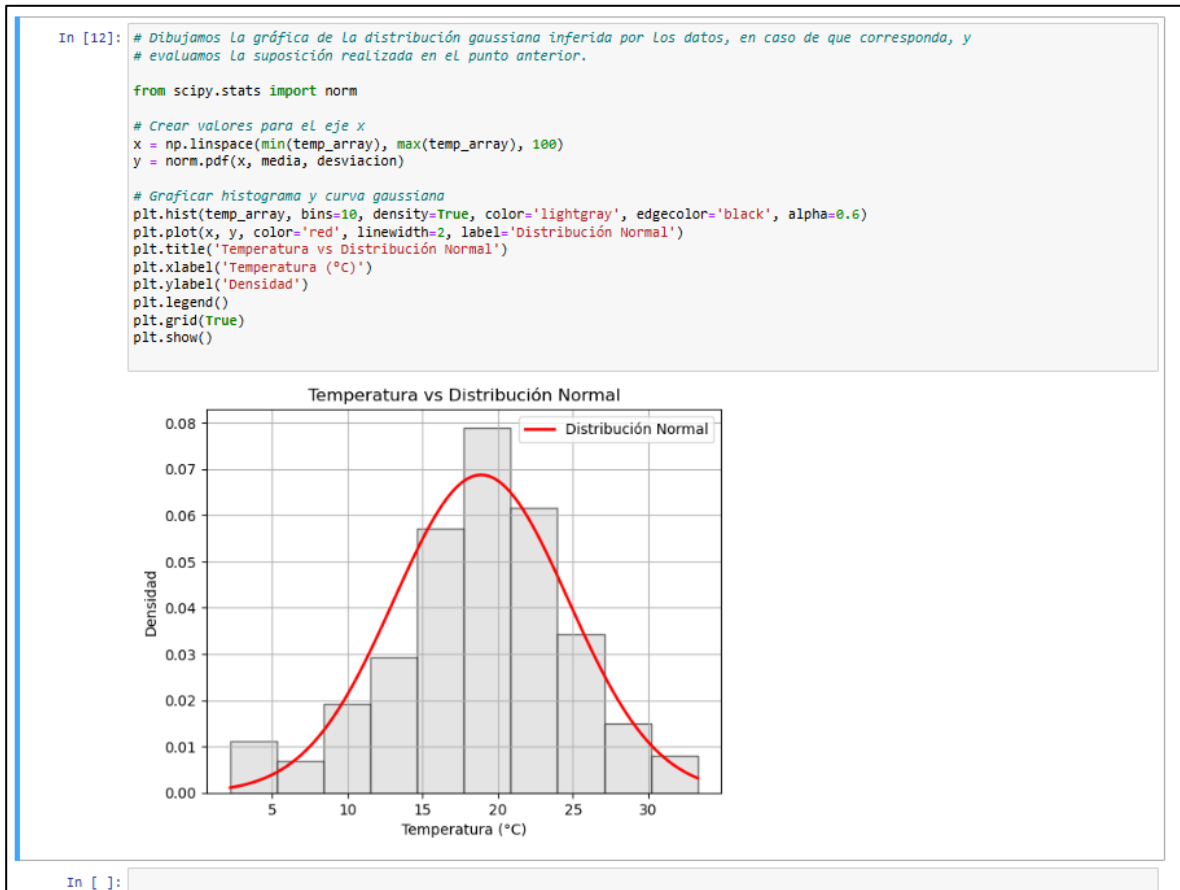
Visualmente se observa que el histograma tiene forma de campana. También podemos usar pruebas estadísticas como Shapiro-Wilk o Kolmogorov-Smirnov si para mayor rigor.

```
In [11]: # Evaluamos si la gráfica obtenida tiene semejanzas con la distribución normal y
# obtenemos los siguientes estadísticos: Media, Mediana y Desviación estándar.
media = np.mean(temp_array)
mediana = np.median(temp_array)
desviacion = np.std(temp_array)

print(f"Media: {media:.2f}")
print(f"Mediana: {mediana:.2f}")
print(f"Desviación estándar: {desviacion:.2f}")

Media: 18.89
Mediana: 19.30
Desviación estándar: 5.80
```

- Dibujamos la gráfica de la distribución gaussiana inferida por los datos, en caso de que corresponda, y evaluamos la suposición realizada en el punto anterior.



La curva roja se ajusta bien al histograma, por lo que podemos asumir que los datos siguen aproximadamente una distribución normal. Esto puede ser útil para aplicar técnicas estadísticas paramétricas.

Conclusiones

El Cálculo Diferencial es un pilar del Modelado Inteligente, las derivadas y gradientes permiten analizar sensibilidad local y optimizar modelos. Algoritmos como la retropropagación y el descenso de gradiente dependen de la diferenciación. El uso de funciones vectoriales y matrices jacobianas fortalece la conexión entre cálculo multivariable y el álgebra lineal, facilitando el análisis de sistemas con múltiples variables.

La Probabilidad es el fundamento del Razonamiento Estadístico, las distribuciones (Bernoulli, Binomial, Gaussiana) modelan incertidumbre y respaldan decisiones. Herramientas como el teorema de Bayes y las reglas de suma/producto permiten evaluar probabilidades condicionales y conjuntas. La probabilidad guía la validación de hipótesis y la selección de modelos adecuados.

El lenguaje de programación Python es un puente entre la teoría y su aplicación

- Herramientas como Jupyter Notebook, `SymPy`, `NumPy`, `Matplotlib` y `Pandas` permiten experimentar con conceptos matemáticos.
- La programación facilita la visualización, optimización y validación de modelos de IA.
- La integración de cálculo, estadística y código es clave para resolver problemas reales con precisión y confiabilidad.

La integración del cálculo, la probabilidad y la programación nos brinda el potencial para que problemas reales en inteligencia artificial sean modelados, examinados y resueltos. Estas ayudas nos proporcionan no solo ideas sobre cómo se comportan los modelos, sino también formas de optimizarlos y validarlos matemáticamente. El dominio de estos fundamentos es vital para crear soluciones inteligentes y confiables.



Liga al código en Github

<https://github.com/luisgg121/ML-Actividad-03.git>